

# CSC 212: Data Structures and Abstractions

## 06: Dynamic Arrays (part 2)

Prof. Marco Alvarez

Department of Computer Science and Statistics  
University of Rhode Island

Fall 2026



### Where we stand?

- Arrays give  $\Theta(1)$  indexing, because memory is contiguous
  - inserting or removing in the middle costs  $\Theta(n)$ , because data must move
- Grow by a constant  $c$  costs  $\Theta(n^2)$  for  $n$  appends
- Open question:
  - is there a growth rule where  $n$  appends cost only  $\Theta(n)$ ?

2

### Grow by doubling

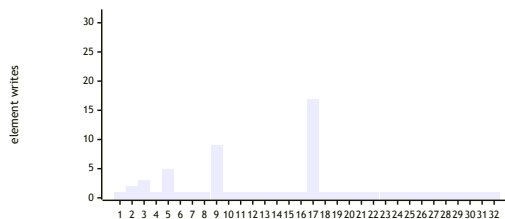
- When full, grow to:  $2 \times \text{capacity}$  (starting from 1)
  - copies get bigger, but they get rarer
  - capacity:  $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow \dots$

|           |                       |                          |
|-----------|-----------------------|--------------------------|
| push 1:   | [1]                   | grow to 1                |
| push 2:   | [1 2]                 | full: copy 1, grow to 2  |
| push 3:   | [1 2 3 _]             | full: copy 2, grow to 4  |
| push 4:   | [1 2 3 4]             | room: no copy            |
| push 5:   | [1 2 3 4 5 _ _ _]     | full: copy 4, grow to 8  |
| push 6-8: | [1 2 3 4 5 6 7 8]     | room: no copies          |
| push 9:   | [1 2 ... 9 _ _ _ _ _] | full: copy 8, grow to 16 |

3

### Cost of each push\_back

- Element writes for each of the first 32 appends
  - copies + the write itself
  - total for 32 appends: 64 writes, approximately **2 per append**



4

## Counting the copies

- Copies happen when size reaches  $1, 2, 4, \dots, 2^k$ , the powers of 2 below  $n$ 
  - ✓ a geometric series
  - ✓ copies =  $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1$
  - ✓ since  $2^k < n$ , we get  $2^{k+1} < 2n$ , so copies  $< 2n$
- Count element copies and writes for  $n$  appends, starting **empty**
  - ✓ ignore cost of allocating/deallocating memory

| append | copies | writes |
|--------|--------|--------|
| 1      | 0      | 1      |
| 2      | 1      | 1      |
| 3      | 2      | 1      |
| 4      | —      | 1      |
| 5      | 4      | 1      |
| 6      | —      | 1      |
| 7      | —      | 1      |
| 8      | —      | 1      |
| 9      | 8      | 1      |
| 10     | —      | 1      |
| ...    | ...    | ...    |
| n-1    | —      | 1      |
| n      | —      | 1      |

$$\begin{aligned}
 T(n) &= n + \sum_{i=0}^{\log_2 n - 1} 2^i \\
 &= n + \left( \frac{2^{\log_2 n} - 1}{2 - 1} \right) \\
 &= n + (n - 1) \\
 &= \Theta(n) \quad \leftarrow \text{cost of adding } n \text{ elements}
 \end{aligned}$$

assume  $n$  is a power of 2

The **amortized cost** of inserting an element is  $\Theta(1)$  and any sequence of  $n$  insertions takes at most  $\Theta(n)$  time in total.

5

## Amortized cost

- Amortized cost is the **average cost per operation**
  - ✓ over a sequence of operations, even if some individual operations are expensive
  - ✓ simple way to analyze cost where occasional costly operations are offset by cheaper ones
- Dynamic arrays with doubling
  - ✓ cost is  $\Theta(1)$  when there is space,  $\Theta(n)$  when growing
  - ✓ over  $n$  appends, the total cost is  $\Theta(n)$
  - ✓ the amortized cost per insertion is  $\Theta(1)$

6

## Q6-1

- A dynamic array doubles when full. Which statement is **false**?

- A single `push_back` is  $O(n)$  in the worst case
- $n$  calls to `push_back` take  $\Theta(n)$  in total
- $n$  calls to `push_back` take  $O(n^2)$  in total
- $n$  calls to `push_back` take  $\Theta(n^2)$  in total
- None of the above

7

## Answer

- D)  $n$  calls to `push_back` take  $\Theta(n^2)$  in total
  - ✓ the others are true
    - A) a call that triggers a grow copies all  $n$  elements
    - B) the tight bound
    - C) true, but **loose**,  $O$  is only an upper bound

8

## Grow by factor

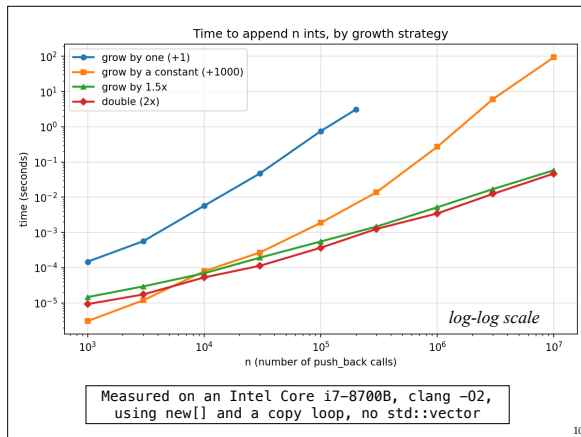
- When full, grow to:  $r \times \text{capacity}$ , for any  $r > 1$ 
  - any factor greater than 1 works
  - the copies still form a geometric series

$$\text{copies} < n \frac{r}{r-1} = \Theta(n)$$

| factor | copies, at most | unused slots, after a grow |
|--------|-----------------|----------------------------|
| 2      | $2n$            | up to 50%                  |
| 1.5    | $3n$            | up to 33%                  |
| 1.123  | $9n$            | up to 11%                  |

*smaller factor: less waste, more copies*

9



10

## Q6-2

- Which growth rule makes  $n$  appends cost  $\Theta(n^2)$ ?

- A. Grow to 1.01 x capacity
- B. Grow to 1.5 x capacity
- C. Grow to capacity + 10,000
- D. Grow to 2 x capacity
- E. None of the above

11

## Answer

- C) Grow to capacity + 10,000

- adding a constant is  $\Theta(n^2)$  cost, whatever the constant

12

## Growth factors in real libraries

- C++ (`std::vector`)
  - ✓ implementation-defined (not in the C++ standard)
  - ✓ grow by 1.5 (MS Visual C++) or 2.0 (g++/clang)
- Java (`ArrayList`)
  - ✓ grow by 1.5 in OpenJDK
- Python (`List`)
  - ✓ grow by ~1.125
- Rust (`std::vec::Vec`)
  - ✓ typically grow by 2

[https://en.wikipedia.org/wiki/Dynamic\\_array](https://en.wikipedia.org/wiki/Dynamic_array)

13

## Shrinking the array

- After many `pop_back` calls, most of the capacity may be empty
  - ✓ should shrink
- First idea, halve the capacity when array is half full
  - ✓ **worst-case** when array is full and alternate between adding and removing
  - ✓ each alternating operation would require resizing the array
- More efficient resizing
  - ✓ half the capacity when the array is **one-quarter** full
- In practice ...
  - ✓ most standard implementations do not automatically shrink capacity
    - avoids performance penalties from frequent resizing
  - ✓ instead, they provide explicit operations like `shrink_to_fit()` that allow the programmer to request size reduction when deemed necessary

14

## Implementation notes

## Implementing append and indexing

```
void DynArray::push_back(int val) {
    if (m_size == m_capacity)
        grow(m_capacity == 0 ? 1 : 2 * m_capacity); // double
    m_data[m_size++] = val;
}

int& DynArray::operator[](size_t i) {
    return m_data[i]; // no check: fast
}

int& DynArray::at(size_t i) {
    if (i >= m_size) // checked: safe
        throw std::out_of_range("DynArray::at");
    return m_data[i];
}
```

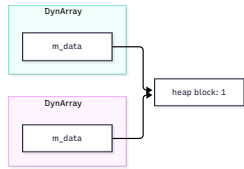
- `operator[]` trusts the caller, at checks bounds (same behavior as `std::vector`)
- Returning `int&` lets the caller write: `a[3] = 7`

16

## The copy problem

- C++ writes a copy constructor for us. It copies each **member**, including the pointer

```
{
    DynArray a;
    a.push_back(1);
    DynArray b = a; // default copy: copies m_data, the address
} // end of scope: ~b frees the block, then ~a frees it again
```



One block, two owners. A *shallow copy* copies the pointer, a *deep copy* copies the data.

17

## Rule of three, rule of five

- A class that owns memory **needs all three, or none**
  - destructor: frees the block
  - copy constructor: `DynArray b = a;` makes a deep copy
  - copy assignment: `b = a;` frees b's block, then makes a deep copy
- Rule of five adds move operations: take over a temporary's block instead of copying it

```
DynArray::DynArray(const DynArray& other) // perform deep copy
: m_data(other.m_data ? new int[other.m_capacity] : nullptr),
  m_size(other.m_size), m_capacity(other.m_capacity) {
    for (size_t i = 0; i < m_size; ++i)
        m_data[i] = other.m_data[i];
}

DynArray& DynArray::operator=(const DynArray& other) {
    if (this != &other) {
        DynArray copy(other); // deep copy first
        std::swap(m_data, copy.m_data); // take the copy's block
        std::swap(m_size, copy.m_size);
        std::swap(m_capacity, copy.m_capacity);
    } // copy's destructor frees our old block
    return *this;
}
```

18

## Q6-3

- An AI assistant wrote this class. It compiles without warnings. What happens at `b = a`?

- The copy constructor runs, so `b` gets a deep copy
- The default copy assignment runs: `b` shares `a`'s block, `b`'s old block leaks, and the block is freed twice
- It does not compile, because `operator=` is missing
- `b = a` moves the data from `a` into `b`
- None of the above

```
class DynArray {
private:
    int* m_data;
    size_t m_size;
    size_t m_capacity;

public:
    DynArray() : m_data(nullptr), m_size(0), m_capacity(0) {}
    DynArray(int val) : m_data(new int[1]), m_size(1), m_capacity(1) {}
    DynArray(const DynArray& other) {
        m_data = new int[other.m_capacity];
        m_size = other.m_size; m_capacity = other.m_capacity;
        for (size_t i = 0; i < m_size; ++i)
            m_data[i] = other.m_data[i];
    }
    void push_back(int val) { // doubles when full, as before
    }

    int main() {
        DynArray a;
        a.push_back(1);
        DynArray b;
        b.push_back(2);
        b = a;
    }
}
```

19

## Answer

- B) The default copy assignment runs: `b` shares `a`'s block, `b`'s old block leaks, and the block is freed twice
  - `DynArray b = a;` is **construction** (copy constructor is called)
  - `b = a;` on an existing object is **assignment**: not written in the code, so C++ copies the members
  - result: a leak (`b`'s old block) and a double free (at the end of `main`)

20

## std::vector from C++ STL

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    size_t last = v.capacity();
    for (int i = 0; i < 1000; ++i) {
        v.push_back(i);
        if (v.capacity() != last) {
            last = v.capacity();
            std::cout << "size " << v.size() << " -> capacity " << last << '\n';
        }
    }
}
```

What is the output?

```
size 1 -> capacity 1
size 2 -> capacity 2
size 3 -> capacity 4
size 5 -> capacity 8
size 9 -> capacity 16
...
size 513 -> capacity 1024
```

<https://en.cppreference.com/w/cpp/container/vector>

21

## Iterator invalidation

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};
    int& first = v[0];
    v.push_back(4);
    std::cout << first << '\n'; // may reallocate: first now dangles
                                // undefined behavior
}
```

What is the output?

After any call that may grow a vector (push\_back, insert, resize, reserve), pointers, references, and iterators to its elements may be invalid. Keep indices instead.

<https://en.cppreference.com/w/cpp/container/vector>

22

## Q6-4

- Fill in the table for a dynamic array that doubles when full and never shrinks. Use  $\Theta$

| Operation                           | Best case | Worst case |
|-------------------------------------|-----------|------------|
| Append 1 element                    |           |            |
| Remove 1 element from the end       |           |            |
| Insert 1 element at index idx       |           |            |
| Remove 1 element at index idx       |           |            |
| Read element at index idx           |           |            |
| Write (update) element at index idx |           |            |

23

## Q6-5

- Run this code, which loop is  $\Theta(n^2)$ ?

```
import time
n = 100000
start = time.time()
array = []
for i in range(n):
    array.append('s')
print(time.time() - start)

start = time.time()
array = []
for i in range(n):
    array = array + ['s']
print(time.time() - start)
```

- A. The first: `append` copies the list every time
- B. The second: `array + ['s']` builds a brand-new list every time
- C. Both: Python lists are linked lists
- D. Neither: both are  $\Theta(n)$  in total
- E. None of the above

24

## Answer

- B) The second: `array + ['s']` builds a brand-new list every time
  - ✓ `array + ['s']` allocates a new list and copies all  $k$  elements into it: that is **grow by one**
  - ✓ **append** grows the list in place, with spare capacity
- ✓ run (Python 3.9, the same machine as before)
  - for  $n = 100,000$ , append takes 0.0075 s and the grow by one method takes 11.7 s (about 1,500x slower)
- ✓ Python lists are dynamic arrays

25

## Designing a dynamic array class in C++

```
#include <cstdlib>
//
// This simplified implementation uses int. A production-quality dynamic array
// would use C++ templates to support arbitrary types, as std::vector<T> does.
// It also would implement the Rule of Three/Five for proper memory management.
//
class DynamicArray {
private:
    int *m_arr;           // pointer to the (internal) array
    size_t m_capacity;    // total number of elements that can be stored
    size_t m_size;        // number of elements currently stored

public:
    DynamicArray();        // constructor (allocates memory)
    ~DynamicArray();       // destructor (frees memory)
    void push_back(int val); // add an element to the end
    void pop_back();        // remove the last element
    void insert(int val, size_t idx); // insert an element at a specific index
    void erase(size_t idx); // remove an element at a specific index
    void resize(size_t len); // change the capacity of the array
    size_t size() const;    // return the number of elements
    size_t capacity() const; // return the capacity
    bool empty() const;    // check if the array is empty
    void clear();          // remove all elements, maintaining the capacity

    // additional methods can be added here
};
```

A class definition specifies the **data members** and **member functions** of the class. The data members are the attributes of the class, and the member functions are the operations that can be performed on the data members. The class definition is a blueprint for creating objects of the class.

26